Jana Dantyszka St, Warsaw, Poland
Khimichna St., Lviv, Ukraine
(+380) 665 0444 27
hello@radency.com

RADENCY

Test Task: Property Rental Management System

📋 Overview

- **Objective:** This technical assessment is to demonstrate your proficiency in ASP.NET Core WebAPI to create a full-stack web application (for frontend you can choose either stay in ASP.NET MVC approach or prepare SPA based on React.js).

- **Roles:** The ideal applicant should possess a good understanding of ASP.NET Core, Entity Framework Core, database design. Use any UI or JavaScript libraries you desire (React.js would be a plus).

- **Evaluation:** We ask that you complete everything in the requirements section. Items outlined in the “bonus” section are completely optional, and your evaluation will not be negatively affected if you choose not to complete them.

🎯 Challenge

Create a full-stack web application for a property management company that accepts rental applications for its apartments. A property manager maintains properties and their units. An applicant applies for a unit and, if approved, is issued a lease. There are only two roles in this system: an applicant and a property manager. There are certain actions that an applicant and a property manager can perform; ensure the permissions in the controllers andeз the UI reflect this.

- **a.** The applicant can create, fill out, submit, and withdraw applications for available units, and correct and resubmit an application that was returned.

- **b.** The property manager maintains properties and units and reviews submitted applications. An approved application issues a twelve-month lease for the unit.

⚙️ Technical Requirements

1. **Build the web application with ASP.NET Core:**
    - Use Clean Architecture or layered architecture.
    - When the validation or any other business rules fail, return a formatted response.
    - Structure and style the project to your liking using best practices.
2. **Create the backend using .NET 10:**
    - Use ASP.NET Identity for User/Role Management.
    - On Start:
        - The database should be created, and migrations applied.
        - The database should be seeded idempotently with lookups, property managers, applicants, properties, units, and applications in every status.
            - Seed your database with data using Bogus for .NET.
    - Add unit tests for business logic.
3. **Use SQL Server / PostgreS:**

- Use Entity Framework Core with code-first database migrations.

💼 Functional Requirements

1. **Users:**
    - Users should be able to sign up, log in, and log out.
        - For convenience, sign-up allows the user to specify if they are an Applicant or a Property Manager.
2. **Properties and Units:**
    - A property manager can add, edit, and remove properties and their units (unit number, bedrooms, monthly rent, unit type) through modals. Applicants can browse available units and start an application for one.
    - Unit Type is a lookup with Active and Inactive values. An inactive value still displays on a unit that already uses it but cannot be selected for any other unit. Enforce this on the server.
    - Approval creates a lease for the unit with a start date and a twelve-month term. A unit whose lease term covers today is not available.
3. **Rental Application:**
    - An application is for one unit and has two sections and a summary:
        - Applicant Information (name, phone, email, current address)
        - Residence History (a list of prior residences, each with an address, landlord name and phone, and move-in and move-out dates)
        - Summary (read-only view of both sections and Submit)

- Residences are added, edited, and removed through a modal.
- An applicant can edit while the application is in a Draft or Returned status; otherwise, every section is read-only. Controllers reject posts that are not allowed.
- At submit and again at approval, reject the API call with an error when the unit has an active lease. Other open applications for that unit are left as they are. The approval check prevents a second lease.

4. **Review:**
   - A property manager opens a submitted application and completes it through a review modal with an outcome of Approve, Return, or Deny and a comment (required for Return and Deny).
   - Application statuses are Draft, Submitted, Returned, Approved, Denied, and Withdrawn. Approved, Denied, and Withdrawn are terminal.
   - The application page shows property managers a history of status changes and review outcomes (who, when, comment).
5. **Application List:**
   - A list of applications filtered by status and property, with the filtering done in the database, not in memory. Applicants see their own applications and property managers see all of them.

🎁 Bonus (Optional)

1. Add paging and sorting to the application list, done in the database, then extract the list into a reusable grid driven by a JSON endpoint that returns the rows and the filtered total, documented with OpenAPI.

2. Add a review queue. A property manager claims a submitted application (Under Review) before completing it, and can release it back to the queue.

3. Add property manager notes that are visible and editable only to property managers and are never rendered or returned to an applicant.

4. Allow a section to be saved even when it fails validation, showing the errors. The Summary lists everything still blocking submission, and Submit stays blocked while any error remains. Define rules once per section and return errors to the field they belong to.

5. Allow more than one applicant on an application. Any applicant on it can view and edit it. Ownership checks apply to all of them.

    - With two applicants editing at once, saves to different sections must not interfere with each other, and when both save the same section, the second save is rejected as stale with a message to reload rather than silently overwriting. No real-time synchronisation is expected.

📌 Considerations & Best Practices

- Any elements not specifically detailed in the guidelines are left to your discretion, and you are encouraged to use your sound judgment in completing the assessment as you see fit.

- Approach this exercise like any other development task, ensuring the code you produce is clean, robust, and ready for a production environment.

- You should follow the standards, conventions, and best practices associated with any frameworks that you use.
- It's important to integrate thoughtful design principles into your work. Be ready to explain the rationale behind your design decisions and the specifics of your implementations.
- The exercise is timed, and you are not rewarded for a delivery; however, **we ask that you return it within 3 days from today's date**. If unforeseen situations demand more time, please let us know your requested delivery date. Failure to deliver on the requested date may result in disqualification.

📦 Deliverables

- 🔗 Once complete, submit a link to your GitHub repository.
- 📄 Include a README.md file with any setup or installation instructions beyond those standard for the technology used in this assessment.
- 📽️ A video recording is required. It must demonstrate the project running locally and meeting all assessment requirements, **as well as include a discussion of** your technical solution (in English).